

ChemoInteract

Complete Documentation — Part 1 of 3

System Overview, Directory Structure, File Reference, Workflow & Datasets

Author:	Ronit	Version:	0.1.0
Framework:	PyTorch 2.0+	Focus:	System Architecture

Contents

1	Project Overview	2
1.1	What is ChemoInteract?	2
1.2	Core Pharmacology Problems Solved	2
1.3	Key Features	2
2	Folder and Package Structure	3
2.1	Directory Tree	3
2.2	Package Sub-Modules	3
3	File-by-File Technical Reference	4
3.1	Core Package Files	4
4	Technology Stack	4
5	Project Workflow and Execution Flow	5
5.1	End-to-End Execution Sequence	5
6	Datasets and Feature Representations	6
6.1	Supported Benchmark Datasets	6
6.2	Feature Storage Format	6
6.3	Negative Sampling Strategy	6

1 Project Overview

1.1 What is ChemoInteract?

ChemoInteract is a modular deep learning library built on PyTorch. It provides a clean, standardized software framework for predicting what happens when two pharmaceutical drugs are administered together. It is a research-grade Python library — not a web application or mobile app.

When a patient takes multiple medications simultaneously, those drugs can interact in complex biological ways. Some interactions cause adverse side effects; others improve therapeutic efficacy. ChemoInteract uses AI to *predict* these interactions computationally *before* conducting expensive wet-lab experiments.

Note

ChemoInteract is structured as a **Python package** (like NumPy or scikit-learn). Users install it, import modules, and invoke functions like `pipeline()` to train and evaluate models.

1.2 Core Pharmacology Problems Solved

Problem	Definition	Clinical Impact
Drug-Drug Interaction (DDI)	Predicting if two drugs interact when co-administered	Prevents adverse drug events in polypharmacy patients.
Polypharmacy Side Effects	Predicting specific side-effects resulting from drug pairs	Identifies non-obvious adverse reactions not seen in single-drug trials.
Drug Synergy	Predicting if two drugs work <i>better together</i> than independently	Critical in oncology to design effective combination chemotherapy regimens.

1.3 Key Features

- **11 Deep Learning Models:** FC networks, GNNs, and Attention models in one repository.
- **5 Benchmark Datasets:** Auto-downloaded, parsed, and cached on disk.

- **One-Line Training:** Unified `pipeline()` replaces hundreds of lines of boilerplate.
- **Modular Design:** Swappable models, datasets, optimizers, and loss functions.
- **Automatic Hardware Acceleration:** CUDA GPU detection with seamless CPU fallback.

2 Folder and Package Structure

2.1 Directory Tree

```

1 ChemoInteract/
2 |-- .git/                <- Git repository version control
3 |-- README.md            <- Overview and quickstart documentation
4 '-- ChemoInteract_Main/  <- Source code directory
5     |-- ChemoInteract/   <- Main Python package
6         |-- __init__.py   <- Package entry point
7         |-- version.py    <- Version string ("0.1.0")
8         |-- constants.py  <- Shared constants (e.g. node features)
9         |-- compat.py     <- TorchDrug/RDKit compatibility patches
10        |-- utils.py      <- Shared tensor & device utilities
11        |-- loss.py       <- Custom loss functions (CASTER)
12        |-- pipeline.py   <- Main pipeline() executor & Result
13        dataclass
14        |-- data/          <- Data loading & batching sub-package
15        |-- '-- models/    <- 11 deep learning model implementations
16        |-- data_cleaning/ <- Preprocessing & cleaning scripts
17        |-- dataset/       <- Pre-processed data JSON/CSV files
18        |-- examples/      <- Demo scripts showing library usage
19        '-- tests/         <- Unit test suite (pytest)

```

2.2 Package Sub-Modules

- **ChemoInteract.data:** Contains dataset loaders (`DatasetLoader`, `DrugCombDB`, `TwoSides`, etc.), mini-batch iterator (`BatchGenerator`), feature containers (`DrugFeatureSet`, `ContextFeatureSet`), and triples wrapper (`LabeledTriples`).
- **ChemoInteract.models:** Contains abstract `Model` base class and all 11 model implementations (`DeepSynergy`, `DeepDDI`, `MatchMaker`, `DeepDDS`, `DeepDrug`, `CASTER`, `GCNBMP`, `EPGCN-DS`, `MR-GNN`, `MHCADDI`, `SSI-DDI`).

3 File-by-File Technical Reference

3.1 Core Package Files

File	Purpose & Functionality
<code>version.py</code>	Defines <code>__version__</code> = "0.1.0". Tracks versioning in experiment results.
<code>constants.py</code>	Dynamically calculates <code>TORCHDRUG_NODE_FEATURES</code> (69 atom features) via dummy molecule initialization.
<code>compat.py</code>	Compatibility layer: patches missing RDKit <code>mplCanvas</code> , adds <code>.to(device)</code> method to TorchDrug <code>PackedGraph</code> , unifies atom feature dictionary keys across versions.
<code>utils.py</code>	Provides <code>resolve_device()</code> , <code>segment_max()</code> , <code>segment_sum()</code> , and numerically stable <code>segment_softmax()</code> .
<code>pipeline.py</code>	Houses <code>Result</code> dataclass and <code>pipeline()</code> function. Orchestrates device resolution, data splitting, model training, loss computation, evaluation, and logging.
<code>loss.py</code>	Implements <code>CASTERSupervisedLoss</code> : a 3-part loss combining binary cross-entropy prediction loss, autoencoder reconstruction loss, and sparse dictionary regularisation.

Warning

Impact of removing `compat.py`: TorchDrug `PackedGraph` objects will fail during GPU tensor transfer (`AttributeError: 'PackedGraph' object has no attribute 'to'`).

4 Technology Stack

Library/Tool	Version	Role in Project
Python	3.8+	Primary programming language
PyTorch	2.0+	Deep learning tensors, autograd, GPU acceleration

Library/Tool	Version	Role in Project
TorchDrug	Latest	Molecular graphs (PackedGraph), GCN layers, readouts
RDKit	Latest	Cheminformatics, SMILES parsing, 256-bit Morgan fingerprints
scikit-learn	1.0+	Metrics (ROC-AUC, MSE, MAE), train/test splitting
Pandas	1.3+	DataFrames for triples and prediction storage
NumPy	1.21+	High-performance array operations
class-resolver	Latest	Dynamic string-to-class resolution for models & loaders
pystow	Latest	Cross-platform directory cache management
tqdm	4.0+	Progress bar visualization during training
tabulate	0.8+	Console output table formatting

5 Project Workflow and Execution Flow

5.1 End-to-End Execution Sequence

1. **Initialization:** User calls `pipeline(dataset="drugcombdb", model="deepsynergy", epochs=100)`.
2. **Device Resolution:** `resolve_device()` checks CUDA availability; assigns `torch.device("cuda")` or `torch.device("cpu")`.
3. **Dataset Loading:** Resolver instantiates loader. Data files (`drug_set.json`, `context_set.json`, `labeled_triples.csv`) are fetched and cached via `@lru_cache`.
4. **Data Splitting:** `LabeledTriples` splits 80% train, 20% test (`random_state=42`).
5. **Batching:** Two `BatchGenerator` objects instantiated for training and testing loops.
6. **Model & Optimizer Setup:** Model instantiated and moved to target device via `model.to(device)`. Adam optimizer initialized ($\alpha = 0.001$).
7. **Training Loop:**
 - i. `BatchGenerator` yields mini-batch `DrugPairBatch`.
 - ii. `optimizer.zero_grad()` resets gradients.
 - iii. `model.unpack(batch)` extracts required input tensors.
 - iv. `model.forward(...)` computes predictions $\hat{y} \in [0, 1]$.

- v. `loss_fn(pred, target)` calculates scalar loss.
 - vi. `loss.backward()` computes autograd gradients.
 - vii. `optimizer.step()` updates model parameters.
8. **Evaluation:** Model set to `model.eval()`. Predictions generated on test set inside `torch.no_grad()`. ROC-AUC score computed.
 9. **Output:** Returns populated `Result` object with training history and metrics.

6 Datasets and Feature Representations

6.1 Supported Benchmark Datasets

Dataset	Task	Loader Type	Context Type	Primary Source
DrugCombDB	Synergy	Remote	Cell lines	Anti-cancer screen
DrugComb	Synergy	Remote	Cell lines	FIMM database
TwoSides	Side-Effects	Remote	Side effect ID	Polypharmacy reporting
DrugbankDDI	DDI	Remote	Interaction type	FDA DrugBank
OncoPolyPharmacology	Synergy	Local (TDC)	Cancer cell lines	O’Neil 2016 screen

6.2 Feature Storage Format

Every dataset directory maintains three standard files:

```

1 dataset/<dataset_name>/
2 |-- drug_set.json          <- {"drug_id": {"smiles": "...", "features":
   [256 floats]}}
3 |-- context_set.json      <- {"context_id": [float vector]}
4 ‘-- labeled_triples.csv   <- columns: drug_1, drug_2, context, label

```

6.3 Negative Sampling Strategy

Database records only provide positive (known) interactions. Negative samples (label = 0.0) are generated via rejection sampling under the **Closed-World Assumption (CWA)**:

any unrecorded drug-drug-context pair is assumed non-interacting.